Demystifying the Optical Gate: State Machine Logic for Finger-Presence Detection

1. Introduction to Optical State Machine Design in Mobile Apps

Mobile photoplethysmography (PPG) monitoring applications process live camera feeds to estimate heart rates non-invasively. However, raw video streams delivered by mobile camera drivers are inherently noisy, susceptible to frame delivery jitter, ambient light fluctuations, and erratic motion artifacts caused by minor user finger adjustments. Feeding unprocessed frame streams directly into signal-filtering algorithms leads to inaccurate pulse calculations, spurious readings, and excessive battery consumption.

To overcome these physical challenges, mobile PPG applications employ an **optical state machine** (or "optical gate"). This architectural pattern acts as an intelligent software gatekeeper, continually analyzing low-level frame metrics to verify that a user's finger is firmly and completely positioned over both the camera sensor and the torch before triggering compute-intensive signal processing algorithms.

The application featured in this guide is built on a cross-platform Flutter framework supporting both Android and iOS devices. It utilizes standard rear-camera video streams and the device's built-in LED torch to capture green-channel PPG signals, convert pixel streams into real-time pulse intervals, and log accepted data to a structured CSV file.

Core Learning Objective: Master how hardware camera metrics—such as pixel chroma, luma, luma variance, and raw green intensity—are systematically translated into predictable software state transitions to build a robust, fault-tolerant optical sensor gate.

Before examining how state decisions are executed, let us inspect the exact physical metrics—mean luma (Y), chroma (U, V), mean green (G), and luma variance—extracted from incoming video frames to drive the optical decision logic.

2. From Camera Pixels to Decision Metrics: Framing the Inputs

To evaluate finger placement without overloading device processing units, the application configures the camera stream at a low resolution, enables the flash torch, waits 500 ms for initial hardware stabilization, locks camera focus and exposure, and samples a centered rectangular **Region of Interest (ROI)** . This ROI spans the middle half of the frame in both horizontal and vertical dimensions. Pixel sampling uses a stride calculated to evaluate at most approximately 40 positions per ROI dimension.

Because Android and iOS expose raw frame buffer formats differently, platform-specific conversion pipelines standardize luma (Y), chroma (U, V), and green (G) values for every sampled pixel within the ROI:

- **Android (YUV420):** Reads the Y plane directly for luma, while subsampled U and V planes provide chroma. Green (G) and red (R) pixel channels are calculated via clamped color transformation formulas: $$G = \text{clamp}(Y - 0.344136(U - 128) - 0.714136(V - 128), 0, 255)$$ $$R = \text{clamp}(Y + 1.402(V - 128), 0, 255)$$
- **iOS (BGRA8888):** Reads blue, green, and red byte channels directly from the frame memory and computes luma and chroma components: $$Y = 0.299R + 0.587G + 0.114B$$ $$U = 128 - 0.169R - 0.331G + 0.5B$$ $$V = 128 + 0.5R - 0.419G - 0.081B$$

A pixel contributes to the skin-tone count when $Y \ge 20$, $70 \le U \le 150$, and $120 \le V \le 195 \text{ or } 240$. The upper threshold $V \le 195$ represented the initial heuristic baseline (v1/master), which was expanded to $V \le 240$ in updated application iterations (v2/v3) to accommodate a broader spectrum of camera hardware sensitivities, skin tones, and torch light intensity variations.
From these converted pixels, the system computes five primary frame metrics to drive presence gating and downstream signal extraction:
Metric Name,Mathematical / Logical Formula,Role in Decision Logic
Mean Luma ($\text{mean } Y$),$\frac{1}{N} \sum_{i=1}^{N} Y_i$,Measures overall frame illumination. Establishes ambient baseline during calibration (≥ 12 required) and verifies minimum light presence.
Skin-Tone Ratio,$\frac{\text{Count of pixels matching skin criteria}}{\text{Total sampled ROI pixels}}$,"Evaluates skin presence ($Y \ge 20$, $70 \le U \le 150$, $120 \le V \le 195 \text{ or } 240$). Requires ≥ 0.15 for detection and ≥ 0.55 for coverage."
Luma Variance,"$\max\left(0, \text{mean}(Y^2) - \text{mean}(Y)^2\right)$","Quantifies light distribution uniformity across the ROI. Low variance indicates a completely covered, diffuse optical path."
Absolute Baseline Luma Change,$,\text{Current Mean } Y - \text{Baseline Mean } Y
Mean Green ($\text{mean } G$),$\frac{1}{N} \sum_{i=1}^{N} G_i$,"Serves as the actual PPG pulse waveform payload. Evaluated, interpolated, filtered, and logged only when the optical gate is in the ACTIVE state."
With these frame metrics defined, we can analyze how the optical gate routes them through its five operational states.

3. Architectural Deep-Dive: The 5 States of the Optical Gate

The optical gate processes frame metrics through five distinct operational states. Each state defines rigid entry conditions, core objectives, and exit criteria to protect downstream signal processing buffers.

- **Calibration Stage (Initial IDLE Phase)**
- **Trigger In:** Launching the measurement screen and receiving the first video frame from the camera stream.
- **Primary Objective:** Sample ambient lighting while the camera lens remains uncovered to construct baseline averages for mean luma and luma variance.
- **Trigger Out:** Accumulating at least 8 frames over an elapsed duration of at least 1.0 second completes calibration, locking the ambient baseline values while remaining in **IDLE** .
- **IDLE State**
- **Trigger In:** Completion of the calibration sequence, loss of finger detection from **DETECTING** , or initial screen initialization.
- **Primary Objective:** Maintain low-overhead monitoring of ambient conditions while enforcing a global 20-second inactivity timeout.
- **Trigger Out:** Meeting the "Finger Detected" criteria (Skin-tone ratio >= 0.15 AND Mean luma >= 12) shifts the machine to **DETECTING** . If 20 seconds elapse without reaching **ACTIVE** coverage, the system terminates the camera stream, turns off the torch, and returns to the home screen with an inactivity timeout notification.
- **DETECTING State**

- **Trigger In:** Transitioning from **IDLE** upon satisfying the initial "Finger Detected" rules.
- **Primary Objective:** Verify that the user's finger fully covers both the lens and flash torch and remains still across a continuous 450 ms temporal confirmation window.
- **Trigger Out:** Satisfying "Looks Covered" criteria continuously for at least 450 ms transitions the state machine to **ACTIVE** . Failing coverage for 3 consecutive debounced frames resets confirmation timers and returns the gate to **IDLE** if finger presence is lost.
- **ACTIVE State**
- **Trigger In:** Accumulating 450 ms of continuous "Looks Covered" status while in **DETECTING** .
- **Primary Objective:** Lock the gate open, canceling the 20-second inactivity timer. Route raw green-channel samples ($\text{mean } G$) into downstream 30 Hz uniform grid linear interpolation, IIR bandpass filtering, BPM peak estimation, and data/heart_rate_samples.csv disk logging.
- **Trigger Out:** Experiencing 3 consecutive coverage-failure frames followed by a persistent removal confirmation timer reaching or exceeding 300 ms shifts the system into **RELEASED** .
- **RELEASED State**
- **Trigger In:** Accumulating at least 300 ms of confirmed removal time during **ACTIVE** , or when the user manually taps "Finish" during an ongoing scan.
- **Primary Objective:** Halt camera streaming and torch illumination, freeze final session statistics (final BPM, signal quality, elapsed duration, and accepted sample count), and display a frozen "Scan Complete" summary UI view.
- **Trigger Out:** User tapping "Done" on the summary screen disposes camera resources and completes the transition back to the home screen.

To observe how these states switch dynamically on a per-frame basis, we must inspect the precise Boolean rules and debouncing mechanics.

4. The Decision Matrix: Step-by-Step Transition Rules

Every incoming video frame is evaluated through two hierarchical Boolean checks: detecting basic skin presence (**Finger Detected**) versus verifying complete optical sealing of the sensor and light source (**Looks Covered**).

Detection vs. Coverage Boolean Rules

- **Finger Detected:** Verifies minimal skin-tone presence and frame illumination: $$\text{Skin-Tone Ratio} \ge 0.15 \quad \text{AND} \quad \text{Mean Luma} \ge 12$$
- **Looks Covered:** Requires full skin-tone coverage, baseline illumination, and significant relative deflection from the calibrated ambient environment. ALL of the following conditions must be satisfied:
- $$\text{Skin-Tone Ratio} \ge 0.55$$
- $$\text{Mean Luma} \ge 12$$
- *Either* of the following baseline deflections:
- $$\text{Absolute Luma Delta} \ge \max(5, \, 0.08 \times \text{Baseline Mean Luma})$$
- $$\text{Current Luma Variance} \le 0.75 \times \text{Baseline Variance}$$

Debouncing, Timers, and Variable Resets

To prevent momentary physical shifts or frame drops from interrupting an ongoing measurement, state transitions enforce debouncing rules using failure counters and state-specific variable resets:

- **State Entry Resets:** Entering **DETECTING** initializes failure_count = 0 and confirmation_timer = 0. Entering **ACTIVE** cancels the 20-second inactivity timeout and resets failure_count = 0 and removal_timer = 0.
- **In DETECTING State:** Experiencing 1 or 2 failed coverage frames increments failure_count but preserves the running 450 ms confirmation timer. On the 3rd consecutive failed frame, confirmation_timer and failure_count reset to zero. If basic finger presence remains, the state stays DETECTING; if finger presence is lost entirely, it reverts to IDLE.
- **In ACTIVE State:** Experiencing 1 or 2 failed coverage frames increments failure_count while active PPG pulse processing continues uninterrupted. On the 3rd consecutive failed frame, the system initiates a 300 ms removal confirmation timer. If coverage is restored before 300 ms elapses, failure_count and removal_timer reset to 0. If poor coverage persists for 300 ms, the machine transitions to RELEASED.

Per-Frame Decision Logic Pseudocode

```
// PER-FRAME EVALUATION AND STATE TRANSITION LOGIC

FINGER_DETECTED = (skin_tone_ratio >= 0.15) AND (mean_luma >= 12)

LOOKS_COVERED = (skin_tone_ratio >= 0.55)
                AND (mean_luma >= 12)
                AND (
                      (abs(mean_luma - baseline_luma) >= max(5,
0.08 * baseline_luma))
                     OR
                     (luma_variance <= 0.75 * baseline_variance)
                   )

IF state IS RELEASED:
    Stay RELEASED // Terminal state; UI displays frozen Scan
Complete summary

ELSE IF NOT calibrated:
    Add frame mean_luma and luma_variance to calibration buffer
    IF elapsed_calibration_time >= 1.0s AND
calibration_frame_count >= 8:
        baseline_luma = average(calibration_luma_buffer)
        baseline_variance = average(calibration_variance_buffer)
        calibrated = TRUE
    Remain IDLE

ELSE IF state IS IDLE:
    IF inactivity_timer >= 20.0s:
        Stop camera stream and torch
```

```
        Return to home screen with inactivity timeout notification
    ELSE IF FINGER_DETECTED:
        state = DETECTING
        failure_count = 0
        confirmation_timer = 0
        IF LOOKS_COVERED:
            Start confirmation_timer

ELSE IF state IS DETECTING:
    IF inactivity_timer >= 20.0s:
        Stop camera stream and torch
        Return to home screen with inactivity timeout notification
    ELSE IF FINGER_DETECTED AND LOOKS_COVERED:
        failure_count = 0
        Start or continue confirmation_timer
        IF confirmation_timer >= 450ms:
            state = ACTIVE
            failure_count = 0
            removal_timer = 0
            Cancel inactivity_timer
            Begin pulse processing & CSV sample logging
    ELSE:
        failure_count = failure_count + 1
        IF failure_count < 3:
            Preserve confirmation_timer
            Stay DETECTING
        ELSE (failure_count >= 3):
            confirmation_timer = 0
            failure_count = 0
            IF FINGER_DETECTED:
                Stay DETECTING
            ELSE:
                state = IDLE

ELSE IF state IS ACTIVE:
    IF FINGER_DETECTED AND LOOKS_COVERED:
        failure_count = 0
        removal_timer = 0
        Stay ACTIVE // Continue processing mean_G payload
    ELSE:
        failure_count = failure_count + 1
        IF failure_count < 3:
            Stay ACTIVE // Preserve rolling sample history &
progress
        ELSE (failure_count >= 3):
            Start or continue removal_timer
            IF removal_timer >= 300ms:
                state = RELEASED
```

```
                Freeze final BPM, signal quality, duration, and
sample count
                Stop camera stream and torch
                Display Scan Complete view
```

Having examined the Boolean rules governing frame analysis, we can now synthesize the broader software design principles demonstrated by this system.

5. Developer Takeaways: Why This Design Matters

Designing real-time computer vision applications for mobile edge hardware requires balancing signal fidelity, battery consumption, and user experience. The state machine pattern utilized in this optical gate highlights three fundamental lessons for mobile systems engineering:

- **Gating Heavy Computation:** Heavy signal processing operations—including 30 Hz uniform grid linear interpolation, two-stage IIR bandpass filtering (0.75–3.5 Hz), rolling peak prominence checks, and file system I/O writes to data/heart_rate_samples.csv—are strictly restricted to the ACTIVE state. Suppressing these calculations during IDLE and DETECTING saves battery life and prevents ambient optical noise from corrupting pulse calculation buffers.
- **Debouncing Sensor Jitter and Handling Stream Interruptions:**
- *Micro-Jitter Debouncing:* Coupling time-based confirmation windows (450 ms confirmation, 300 ms removal) with a 3-consecutive-frame debouncing threshold creates a temporal low-pass filter. This prevents UI flickering and avoids resetting active session progress when users alter their grip slightly or experience minor camera frame delivery variations.
- *Stream Interruption Recovery:* If hardware camera frame drops exceed 100 ms during an ACTIVE session, the pulse processor handles the stream gap by discarding its interpolation anchor, filter states, rolling waveform history, accepted intervals, and current BPM. It re-anchors the 30 Hz uniform grid at the incoming sample without interpolating across the gap, preserving the overarching active session progress timer while safeguarding signal processing integrity against step-discontinuities.
- **Calibration-Relative Thresholding:** Hardcoded, absolute brightness thresholds fail when deployed across diverse smartphone hardware, varying skin phototypes, and unpredictable ambient lighting. Evaluating runtime frame metrics relative to baseline values established during initial calibration—such as requiring absolute luma deltas of $\ge \max(5, \, 0.08 \times \text{baseline luma})$ or luma variance reductions to $\le 75\%$ of baseline variance—allows the application to dynamically adapt to varying sensor gain curves and torch intensities. Furthermore, coupling this with safety fallbacks like the 20-second inactivity timeout ensures the application safely degrades when coverage cannot be established.

By structuring raw hardware inputs into a debounced, calibrated state machine, mobile developers can transform unconditioned camera feeds into highly resilient, production-ready computer vision pipelines.